Embedding Migration Framework

A tool that lets a company switch from an old embedding model to a new one **without re-embedding their entire corpus** — saving most of the cost, compute, and time.

---

1. The Problem

An embedding model is like a **translator**: it turns every document into a string of numbers (a *vector*). Search, recommendations, and RAG all work by comparing these vectors.

The catch: **every model speaks its own number-language.** Each model places vectors in its own *vector space* — different dimensions, different geometry, different distance assumptions. So a vector made by Model A is meaningless to Model B.

This means when a company wants to upgrade to a newer, better embedding model, all of their old stored vectors instantly become useless to the new model — like a library where every book is written in a language nobody reads anymore.

Today, the only fix is to re-embed the whole corpus with the new model. At scale (millions to billions of documents), that is:

- **Expensive** — huge amounts of API calls or GPU time.
- **Slow** — long migration windows.
- **Risky** — you need the original source text, and many teams never kept it.

Because this is so painful, companies stay locked into outdated models long after better ones exist.

---

2. What We're Solving

> **Let a company migrate to a new embedding model without re-embedding the whole corpus — cutting migration cost and time by roughly 95% while keeping search quality high.**

We are **not** claiming "zero re-embedding." We re-embed a tiny sample, learn from it, then cheaply transform the rest.

---

3. The Core Idea (in plain language)

Instead of re-translating every book, we **teach a small "translator" that converts old-language numbers into new-language numbers.**

We do this by showing it examples. For a small handful of documents, we line up the old version and the new version side by side. From enough of these matched pairs, a simple program can learn the pattern:

```
f(old_vector) ≈ new_vector
```

Once we have that learned function (f) (the **mapper**), we run all the *other* documents' old vectors through it — instantly and almost for free — instead of re-embedding them properly.

The mapped vectors aren't *perfect* copies of the real new vectors, but they're **close enough** that search still works well.

---

4. How It Works — Step by Step

Here's the full flow, using a concrete example: **a company with 1,000,000 documents.**

Step 1 — Sample a small subset

Pick about **1–5%** of the corpus at random (say 30,000 documents). We already have their old vectors stored. We run *only these* through the new model to get their new vectors.

Result: **30,000 matched pairs** — "old looks like this, new looks like this."

> Note: we never need the *old* model here, because the old vectors are already saved. We only run the new model, and only on the sample.

Step 2 — Train the mapper

Feed the 30,000 pairs to a small program that learns the pattern linking old to new. This is the **mapper**. Training takes seconds on a normal laptop.

The other **970,000 documents are never touched** in this step.

Step 3 — Check the quality (the confidence gate)

Before trusting the mapper, we hold back some sample pairs it never saw during training and test on them: translate the old vector, then measure how close the result is to the *real* new vector.

We turn this into a single **confidence number**, e.g. *"search will retain about 92% of full re-embedding quality."*

- If the number is good → proceed.
- If it's bad → **stop and warn the company.** Better to know before committing than after.

This gate is what makes the tool trustworthy and sellable — we can quote expected quality **before** anyone commits.

Step 4 — Transform everything (instant cutover)

Run **all 1,000,000 old vectors** through the mapper. This is just light math, not the heavy new model, so it's instant and nearly free.

Load the results into the search index. **The company is now fully switched to the new model.** Cutover can happen in minutes.

These mapped vectors are the permanent answer — not a placeholder.

---

5. What We Deliberately Left Out (and why)

An earlier version of this idea had a *Step 5*: slowly re-embed the real documents in the background and swap the approximations for perfect ones over time.

We removed it on purpose.

If you eventually re-embed all documents anyway, you've saved **nothing** — you've just spread the original cost out over time. That directly contradicts the entire goal of saving cost and compute.

So the framework ends cleanly at Step 4. The approximate vectors are kept **forever**. This also makes the pitch sharper and fully honest:

> **"Pay to re-embed 3% of your documents, get ~92% of the search quality, and keep it forever."**

No asterisk, no hidden second bill.

---

6. The Steady State (after migration)

A real corpus doesn't freeze — new documents keep arriving. Those new ones are embedded **directly** with the new model (cheap, because it's just the incoming trickle, not the back-catalog).

So the final library is a **permanent mix**:

- **Translated-old vectors** — the migrated back-catalog.
- **True-new vectors** — documents added after the switch.

This is fine: both live in the new model's space, so search works across the whole mix.

---

7. The Mechanism in a Bit More Detail

The mapper

The simplest and best place to start is a **linear map** (ridge regression). "Training" is essentially solving one matrix equation:

```
W = (XᵀX + λI)⁻¹ XᵀY        X = old vectors, Y = new vectors
f(x) = normalize( (x − μ_x) · W + μ_y )
```

- **No GPU, no training loop** — milliseconds on a laptop.
- The learned object is tiny: a matrix `W` plus two mean vectors. Easy to save and reload.
- It naturally handles **dimension mismatch** (e.g. 768 → 1536), because `W` is shaped `d_old × d_new`.

Three details that quietly make or break quality:

1. **Mean-center** both old and new vectors before fitting (and store the means).
2. **L2-normalize** the output if the new model uses cosine similarity.
3. If a linear map isn't accurate enough (judged on the validation slice in Step 3), upgrade to a **small MLP** (1–2 hidden layers). Don't reach for the MLP first — with a tiny sample it overfits easily.

Query-time consistency (important, easy to get wrong)

After cutover, the corpus vectors are *approximate* new vectors, but incoming **queries are embedded directly** with the new model (real new vectors).

Do they match? **Yes** — because the mapper's whole job was to push old vectors *into* the new model's space. So query (true-new) and corpus (approx-new) share a space and are comparable.

Do not map the query. Embed the query with the new model. Mapping the query would be a bug.

---

8. What the Company Must Provide

They provide	Why
The old stored vectors	The bulk of the corpus — we reuse these, never recompute them.
The new model (as a callable: `text → vector`)	To embed the sample and future incoming documents.
Source text for the sample only (1–5%)	To create the matched training pairs in Step 1.

What they **don't** need: the old model, and the source text for the other 95–99%.

What we produce: a **trained mapper**, a **confidence report**, and the **transformed index**.

---

9. Where We Might Face Blockers

Blocker	Detail
Approximation loss	Mapped vectors aren't the real new vectors, so there's some search-quality drop.
Model dissimilarity	Works well when old and new models are *similar*. If they're very different, the old vectors may not contain information the new model needs — and **you can't recover what was never encoded.**
Need training pairs	Requires source text (or the saved sample) so we can build the old↔new pairs.
Quality measurement	It's easy to build a mapper that *looks* fine but tanks real search quality. Rigorous benchmarking (Step 3) is mandatory.

The honest core limitation: **the mapper can only rework information that's already in the old vectors.** If the new model is better because it captures distinctions the old model threw away, no mapper can invent that back. This is why the method shines when models are similar and weakens when they're very different — and why the confidence gate exists.

---

10. Success Metric

> **Retrieval recall@k of mapped vectors vs. true new vectors, on a held-out query set.**

This single number is the project's credibility. Compare three things:

- **Ceiling** — true new vectors → `recall@k_max`
- **Our method** — mapped vectors → `recall@k_mapped`
- **Do-nothing** — keep the old model → `recall@k_old`

We must beat *do-nothing*, and "quality retained" = `recall@k_mapped / recall@k_max`.

(You can also report cosine similarity of `f(old)` vs true-new — but **never** let cosine stand in for recall. A mapper can look great on cosine and still hurt search.)

---

11. Positioning

A **migration accelerator**, not a re-embedding eliminator.

> Pay to re-embed a small sample, get most of the quality, switch over instantly, and keep it permanently.

Honest, sellable, and scalable.

---

12. Existing Landscape

- **No mainstream plug-and-play product** does mapping-based migration today — this is the gap we fill.
- The underlying technique exists in research: cross-lingual embedding alignment, **relative representations** (Moschella et al.), and **model stitching**.
- Vector databases (Pinecone, Weaviate, Qdrant) handle re-indexing but offer **no mapping-based migration feature.**